Khulna University of Engineering & Technology

Department of Computer Science and Engineering

FPGA Based Smart Pomodoro Timer

Target Platform	Digilent Basys 3 (Artix-7 XC7A35T-CPG236)
Product Family	Artix-7
Library	xil_defaultlib
Simulator	Vivado Simulator
HDL	Verilog
Design Tool	Xilinx Vivado - Synthesis, Implementation, Bitstream Generation
Source Files	pomodorovthree.v | pomodorovthree.xdc

Submitted By

MD. FARHAN HOSSAIN
Student ID: 2407019

NABEEL SARWAR TAHMEED
Student ID: 2407013

Submitted To

DR. MD. MILON ISLAM
Assistant Professor, Dept. of CSE, KUET

NABIL FAIYAZ SADI
Lecturer, Dept. of CSE, KUET

Table of Contents

1. Introduction 2

1.1 Project Overview 2

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Scope 2

2. Detailed Methodology 3

2.1 Hardware and Software Requirements 3

2.2 Design Decisions 3

2.3 System Architecture 3

2.4 Logic Explanation with Code Snippets 5

Duration Selector 5

Debounce Logic 5

Second Tick Generator 5

Main FSM — State Transitions 6

DONE State — Auto Session Transition 7

7-Segment Display Controller 7

LED Status Panel 8

Constraint File (pomodorovthree.xdc) 8

2.5 Implementation Challenges 9

3. Conclusion 10

3.1 Summary 10

3.2 Limitations 10

3.3 Future Work 10

3.4 Final Thoughts 10

4. System Flow Chart 11

List of Figures & Tables

Figure 1 — FPGA Based Smart Pomodoro Timer — System Flow Chart 11

Table 1 — Hardware and Software Requirements 3

Table 2 — Implementation Challenges and Resolutions 9

1. Introduction

1.1 Project Overview

This project started out of a common study problem — losing track of time and getting distracted too easily. Using a phone timer makes things worse since picking up the phone breaks focus. The idea was to build a dedicated hardware timer that sits on the desk and does exactly one job, with no screen to swipe and no notifications.

The Pomodoro Technique was chosen as the basis. The classic method works in 25-minute focused sessions followed by 5-minute breaks, with a 15-minute long break after every four sessions. A 52/17 variant was also added — 52 minutes of deep work followed by a 17-minute break — which suits heavier material that benefits from longer uninterrupted stretches. The project is implemented in Verilog HDL on the Digilent Basys 3 FPGA board.

1.2 Problem Statement

Manually tracking focused work and break intervals is inefficient and prone to distraction, especially when relying on smartphones. A hardware-based solution removes digital distraction entirely. The timer runs autonomously, indicates session status via LEDs, counts distractions, and tracks completed Pomodoro cycles without requiring any screen interaction.

1.3 Objectives

- Accurately count 25-minute work sessions, 5-minute short breaks, and 15-minute long breaks after every four Pomodoro cycles.
- Support an alternative 52/17 method — sw[2] UP selects 52-min work / 17-min break; sw[2] DOWN uses classic 25/5.
- Implement automatic session transitions (work → break → work) without any manual intervention.
- Display remaining time as MM:SS on the 4-digit 7-segment display; blinking colon indicates RUNNING state.
- sw[1] (Demo Mode): When HIGH, each timer tick fires 60× faster — compresses a full session to seconds for quick hardware testing.
- sw[3] (Stats Mode): Toggles the 7-segment display between time view (MM:SS) and counter view (Distractions : Pomodoros).
- Track distraction count (btnC press during work = distraction) and completed Pomodoros on LED[11:6] and LED[5:0] respectively in binary.
- Keep the interface minimal: two buttons (btnC = play/pause, btnL = full reset) and three active slide switches.

1.4 Scope

The project targets the Basys 3 specifically, with the XDC constraint file written for its pin layout and 100 MHz oscillator. The FSM and timer logic can be adapted to other Artix-7 boards with only pin assignment changes. It is a single-user tool for focused personal study sessions.

2. Detailed Methodology

2.1 Hardware and Software Requirements

Category	Details
FPGA Board	Digilent Basys 3 (Artix-7 XC7A35T-CPG236)
Product Family	Artix-7
Library	xil_defaultlib
Simulator	Vivado Simulator
Inputs	btnC play/pause | btnL reset | sw[1] demo | sw[2] method | sw[3] stats
Outputs	4-digit 7-segment display + decimal point, 16 onboard LEDs
HDL	Verilog
Design Tool	Xilinx Vivado — synthesis, implementation, bitstream generation

2.2 Design Decisions

Why Verilog over VHDL? — Verilog syntax is closer to C, making it faster to prototype without slowing down on verbose syntax while the logic itself was complex.

Why an FSM? — A flat sequential approach became hard to manage when implementing pause-resume behavior. An FSM maps each state (IDLE, RUNNING, PAUSED, DONE) to a defined behavior, makes transitions explicit, and makes each bug traceable to a specific state.

Why Pomodoro? — The technique directly addresses the problem of maintaining focus during long study sessions. The distraction counter was added because tracking interruptions builds awareness of focus quality over time.

2.3 System Architecture

The design is a single Verilog module (pomodoro) with several internal logic blocks. The module declaration and parameters are shown below, followed by a description of each subsystem.

pomodorovthree.v — Module Declaration & Parameters

```
// Module: pomodoro | Target: Basys 3 (Artix-7 XC7A35T) | Clock: 100 MHz
module pomodoro (
    input  wire        clk,        // 100 MHz system clock
    input  wire        btnC,       // Play / Pause (center button)
    input  wire        btnL,       // Reset (left button)
    input  wire [3:0]  sw,         // sw[1]=demo sw[2]=method sw[3]=stats
    output reg  [6:0]  seg,        // 7-seg cathode {G,F,E,D,C,B,A}
    output reg  [3:0]  an,         // Anode select (active LOW)
    output wire        dp,         // Decimal point / colon
    output reg  [15:0] led         // 16 status LEDs
);

localparam CLK_FREQ     = 100_000_000;
localparam CLASSIC_WORK  = 25 * 60;   // 1500 s
localparam CLASSIC_BREAK =  5 * 60;   //  300 s
localparam LONG_BREAK    = 15 * 60;   //  900 s
localparam METHOD_WORK   = 52 * 60;   // 3120 s
localparam METHOD_BREAK  = 17 * 60;   // 1020 s
localparam DEBOUNCE_TICKS = 2_000_000; // 20 ms @ 100 MHz
```

• **Clock Divider / Tick Generator:** 27-bit counter generates a 1 Hz tick (or 60 Hz in demo mode). Gated to ST_RUNNING — counter resets to zero in all other states.

• **Debounce Logic:** Independent 20 ms stability counters for btnC and btnL. Button must hold same value for 2,000,000 cycles before the edge is accepted.

• **Main FSM:** Four states — IDLE, RUNNING, PAUSED, DONE. Handles all session transitions and counter updates.

• **Timer Logic:** 14-bit timer_sec register decrements on each sec_tick. Loaded from work_dur or break_dur depending on phase and method switch.

• **Long Break Logic:** 3-bit session_in_set counter tracks work sessions 0–3. After the fourth session, is_long_break flag forces break to 15 minutes.

• **Distraction and Pomodoro Counters:** Both 6-bit (0–63). Distractions increment on btnC during work phase. Pomodoros increment on natural work completion.

• **7-Segment Display:** 17-bit refresh counter multiplexes four digits at ~1 ms each. MM:SS in normal mode, DD:PP in stats mode. Colon blinks at 1 Hz when running.

• **LED Status Panel:** LED[15:12] = Work/Break/Running/Paused indicators. LED[11:6] = distraction count (binary). LED[5:0] = Pomodoro count (binary).

2.4 Logic Explanation with Code Snippets

Duration Selector

Combinational block picks work and break durations based on sw[2] (method) and is_long_break flag:

Work / Break Duration Selector

```
// Always-block: combinational — runs whenever inputs change
always @(*) begin
    if (sw_method) begin         // sw[2] HIGH = 52/17 method
        work_dur  = METHOD_WORK[13:0];   // 52 min
        break_dur = METHOD_BREAK[13:0];  // 17 min
    end else begin               // sw[2] LOW = classic 25/5
        work_dur  = CLASSIC_WORK[13:0];  // 25 min
        if (is_long_break)
            break_dur = LONG_BREAK[13:0];    // 15 min (4th session)
        else
            break_dur = CLASSIC_BREAK[13:0]; // 5 min
    end
end
```

Debounce Logic

Applied to both btnC and btnL. Waits 20 ms of stable input before accepting a rising edge:

Debounce — btnC (same pattern for btnL)

```
// 20 ms stability window at 100 MHz = 2,000,000 cycles
always @(posedge clk) begin
    btn_c_edge <= 1'b0;
    if (btnC == btn_c_prev) begin
        if (db_ctr_c == DEBOUNCE_TICKS[20:0]) begin
            if (btnC != btn_c_clean) begin
                btn_c_clean <= btnC;
                if (btnC == 1'b1) btn_c_edge <= 1'b1; // clean rising edge
            end
        end else db_ctr_c <= db_ctr_c + 1'b1;
    end else begin
        db_ctr_c  <= 21'd0;    // reset on change
        btn_c_prev <= btnC;
    end
end
```

Second Tick Generator

Generates 1 Hz tick (or 60 Hz in demo mode). Counter only runs in ST_RUNNING:

Tick Generator | sw[1] = Demo Mode (60x faster)

```
// sec_limit switches between 1s and 1/60s based on demo switch
wire [26:0] sec_limit = sw_demo ? DEMO_SEC[26:0] : ONE_SEC[26:0];

always @(posedge clk) begin
    sec_tick <= 1'b0;
    if (state == ST_RUNNING) begin
        if (tick_ctr >= sec_limit - 1) begin
            tick_ctr <= 27'd0;
            sec_tick <= 1'b1;  // one-cycle pulse every second
        end else tick_ctr <= tick_ctr + 1'b1;
    end else tick_ctr <= 27'd0;  // reset outside RUNNING
end
```

Main FSM — State Transitions

Controls all state transitions. btnL always resets everything to IDLE regardless of current state:

Main FSM — State Transitions

```
// State encoding: ST_IDLE=0, ST_RUNNING=1, ST_PAUSED=2, ST_DONE=3
always @(posedge clk) begin
    if (btn_l_edge) begin         // btnL: global reset
        state <= ST_IDLE;  phase <= PHASE_WORK;
        timer_sec <= work_dur;  distractions <= 0;  pomodoros <= 0;
        session_in_set <= 0;  is_long_break <= 0;
    end else case (state)
        ST_IDLE:    if (btn_c_edge) state <= ST_RUNNING;
        ST_RUNNING: if (btn_c_edge) begin
                        state <= ST_PAUSED;
                        if (phase==PHASE_WORK) distractions++;
                    end else if (sec_tick && timer_sec==0)
                        state <= ST_DONE;
                    else if (sec_tick) timer_sec <= timer_sec - 1;
        ST_PAUSED:  if (btn_c_edge) state <= ST_RUNNING;
        ST_DONE:    // auto-transition handled below
    endcase
end
```

2.4 (continued) — DONE State & Display Logic

DONE State — Auto Session Transition + Long Break Logic

DONE State — Auto Session Transition + Long Break Logic

```
ST_DONE: begin
    if (phase == PHASE_WORK) begin
        if (pomodoros < 63) pomodoros <= pomodoros + 1;
        if (session_in_set == 3'd3) begin   // 4th session done
            session_in_set <= 0;
            is_long_break  <= 1'b1;          // trigger 15-min break
        end else begin
            session_in_set <= session_in_set + 1;
            is_long_break  <= 1'b0;
        end
        phase <= PHASE_BREAK;
        state <= ST_RUNNING;     // auto-start break
    end else begin               // break finished
        phase <= PHASE_WORK;
        is_long_break <= 1'b0;
        state <= ST_RUNNING;     // auto-start work
    end
end
// Sync timer_sec in same cycle as DONE transition
if (state == ST_DONE)
    timer_sec <= (phase==PHASE_WORK && session_in_set==3)
                  ? LONG_BREAK[13:0]
                  : (phase==PHASE_WORK ? break_dur : work_dur);
```

7-Segment Display Controller

Multiplexed over four digits. sw[3] toggles between time and stats display:

7-Segment Multiplexer | sw[3] = Stats Mode

```
// 17-bit refresh counter — switches digit every ~1 ms
always @(*) begin
    an = 4'b1111;  dp_reg = 1'b1;
    case (digit_sel)
        2'd3: begin  an=4'b0111;  // AN3 leftmost
            digit_val = sw_stats ? dist_tens : mm_tens[3:0]; end
        2'd2: begin  an=4'b1011;  // AN2
            digit_val = sw_stats ? dist_ones : mm_ones[3:0];
            if (!sw_stats && state==ST_RUNNING) dp_reg=~blink_reg; end
        2'd1: begin  an=4'b1101;  // AN1
            digit_val = sw_stats ? pomo_tens : ss_tens[3:0]; end
        2'd0: begin  an=4'b1110;  // AN0 rightmost
            digit_val = sw_stats ? pomo_ones : ss_ones[3:0]; end
    endcase
end
// Common-anode decoder (0=ON, 1=OFF)
4'd0: seg=7'b100_0000;  4'd1: seg=7'b111_1001;  // 0, 1
4'd8: seg=7'b000_0000;  4'd9: seg=7'b001_0000;  // 8, 9
```

LED Status Panel

LED Status Panel

```
always @(*) begin
    led = 16'b0;
    // LD15-LD12: Session status indicators
    led[15] = (phase == PHASE_WORK);              // Work active
    led[14] = (phase == PHASE_BREAK);             // Break active
    led[13] = (state == ST_RUNNING) & blink_reg;  // Running — blinks 1 Hz
    led[12] = (state == ST_PAUSED);               // Paused — solid
    // LD11-LD6: Distraction count (6-bit binary, max 63)
    led[11:6] = distractions[5:0];
    // LD5-LD0: Completed Pomodoro count (6-bit binary, max 63)
    led[5:0]  = pomodoros[5:0];
end
```

Constraint File

pomodorovthree.xdc — Key Constraints

```
## Clock — 100 MHz onboard oscillator
set_property -dict { PACKAGE_PIN W5  IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 [get_ports clk]

## Switches: sw[1]=demo  sw[2]=method  sw[3]=stats
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports {sw[1]}]
set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports {sw[2]}]
set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports {sw[3]}]

## Buttons
set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports btnC]
set_property -dict { PACKAGE_PIN W19 IOSTANDARD LVCMOS33 } [get_ports btnL]

## 7-Segment Anodes (active LOW)
set_property -dict { PACKAGE_PIN U2  IOSTANDARD LVCMOS33 } [get_ports {an[0]}]
set_property -dict { PACKAGE_PIN W4  IOSTANDARD LVCMOS33 } [get_ports {an[3]}]

## Configuration
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO       [current_design]
```

2.5 Implementation Challenges

Challenge	Problem	Resolution
1. Button Bouncing	No debounce caused a single btnC press to cycle FSM through 2–3 states.	Added a 20 ms stability counter (2,000,000 cycles at 100 MHz) per button.
2. Timer Too Fast	tick_ctr was not gated to RUNNING, so sec_tick fired during IDLE/PAUSED.	Fixed by resetting tick_ctr to zero outside ST_RUNNING.
3. 7-Seg Display Mirrored	Anode mapping was reversed — digits appeared in wrong positions. Also confused common-anode (0=ON) with common-cathode.	Corrected the anode index mapping and verified segment polarity against the Basys 3 reference manual.
4. FSM Stuck in DONE	timer_sec loaded before break_dur updated with new is_long_break value.	Fixed by explicit synchronization inside DONE that reads session_in_set directly.
5. Long Break Not Triggering	is_long_break was set one cycle after break_dur was already read.	Fixed by setting both flag and timer_sec override in the same clock cycle.
6. 52/17 Break Wrong	Combinational block ran classic break calc before checking sw_method.	Reordering so sw_method is checked first resolved the wrong duration.
7. Constraint File Mismatch	XDC net names did not match module ports. Vivado silently ignores mismatches.	Cross-checked every port name line by line in the XDC.
8. Wrong Device Target	Project created for wrong Artix-7 part. Bitstream built but board refused to program.	Fixed by recreating project with xc7a35tcpg236-1.
9. Lab Time Limits	Bitstream generation takes 10–20 min; lab board time was limited.	Workaround: 7–8 bitstreams pre-built at home, brought on USB. Features added step by step.
10. Testbench Timescale	Vivado waveforms did not match — demo-mode divisor tested at wrong timescale.	Adjusting the timescale parameters made FSM transitions match expectations.

3. Conclusion

3.1 Summary

The project successfully implemented a functional Pomodoro Timer on the Digilent Basys 3 FPGA. The final version accurately manages work and break sessions including the 15-minute long break after every four Pomodoro cycles, and supports the 52/17 method via sw[2]. The 7-segment display shows remaining time with a blinking colon, and the LED panel provides binary feedback on session status, distraction count, and completed Pomodoro count. All stated objectives were met.

3.2 Limitations

- **Fixed Durations:** Work and break times are localparam values — any change requires editing the source and regenerating the bitstream.
- **No Persistence:** Distraction and Pomodoro counts reset on power cycle or btnL press. No non-volatile memory is integrated.
- **Limited UI:** Interface is buttons and switches only — no way to visually confirm sw settings beyond LED[15:12] status indicators.
- **Single User:** Designed for one user with one active session at a time.
- **52/17 Long Break Disabled:** Long break logic is bypassed in 52/17 mode since the two methods have different session structures.

3.3 Future Work

- **Configurable Durations:** Allow users to set custom work/break durations via switches without reprogramming.
- **Non-Volatile Storage:** Integrate EEPROM or Flash to retain session statistics across power cycles.
- **Audio Alerts:** Add a buzzer output for session-transition notifications using the Basys 3 audio interface.
- **LCD/OLED Display:** Replace the 7-segment display for richer output including current mode and settings.
- **Real-Time Clock Module:** Integrate an RTC so the timer is independent of FPGA power state.

3.4 Final Thoughts

Getting the constraint file right, pre-building multiple bitstreams to work around limited lab board time, debugging FSM transitions without a working testbench — the process had more friction than expected. But seeing the 7-segment display count down correctly for the first time in the lab, with the LEDs updating and the colon blinking, made it worthwhile. The project demonstrates that FPGA hardware can be effectively applied to practical productivity tools beyond purely academic exercises.

4. System Flow Chart

The diagram below illustrates the complete operational flow of the Pomodoro Timer FSM, covering all session transitions, long-break logic, distraction/Pomodoro counter updates, and the global btnL reset path.

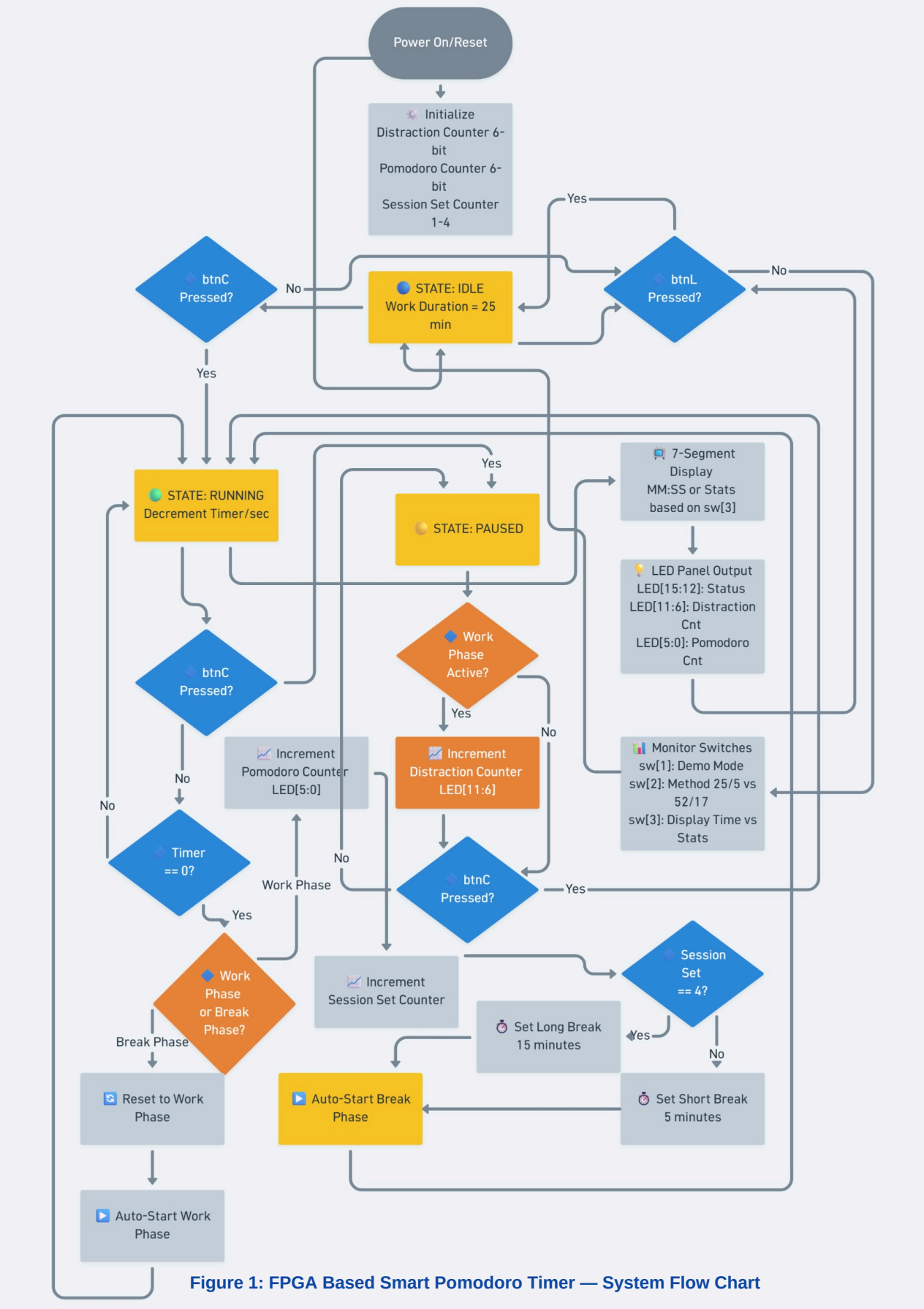

Figure 1: FPGA Based Smart Pomodoro Timer — System Flow Chart